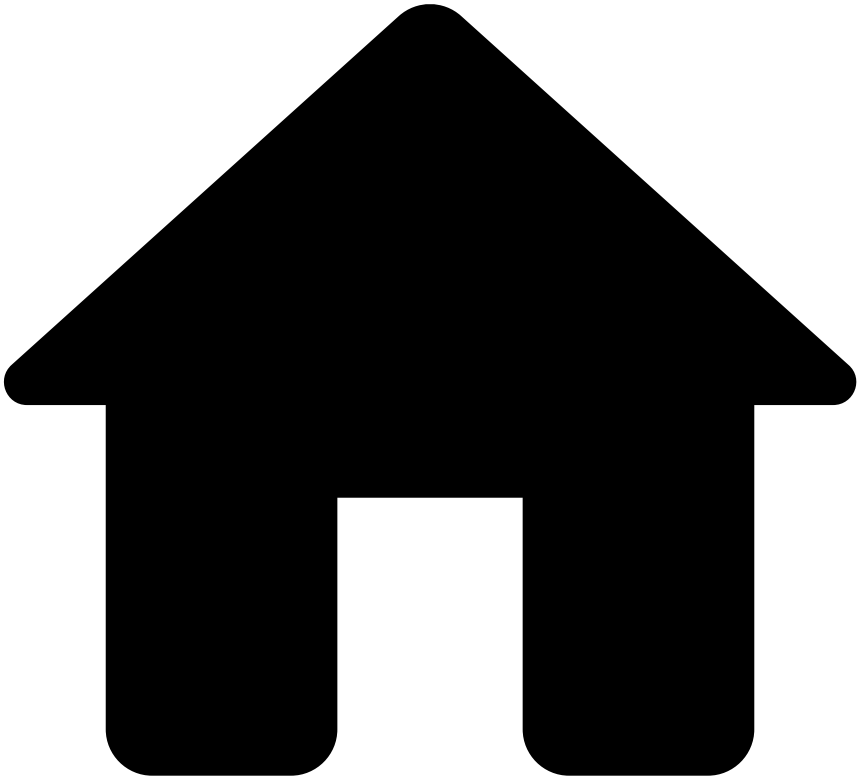
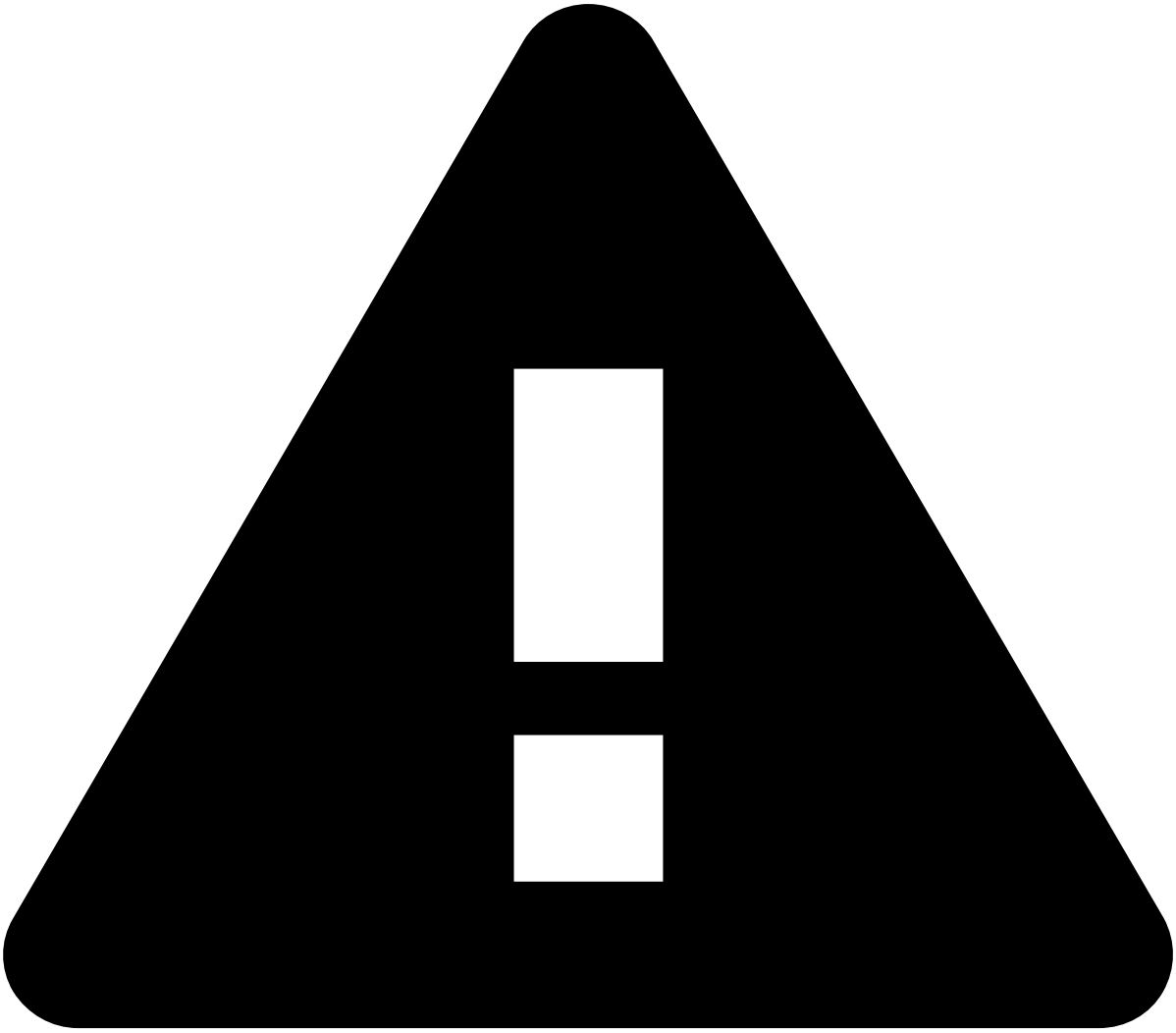


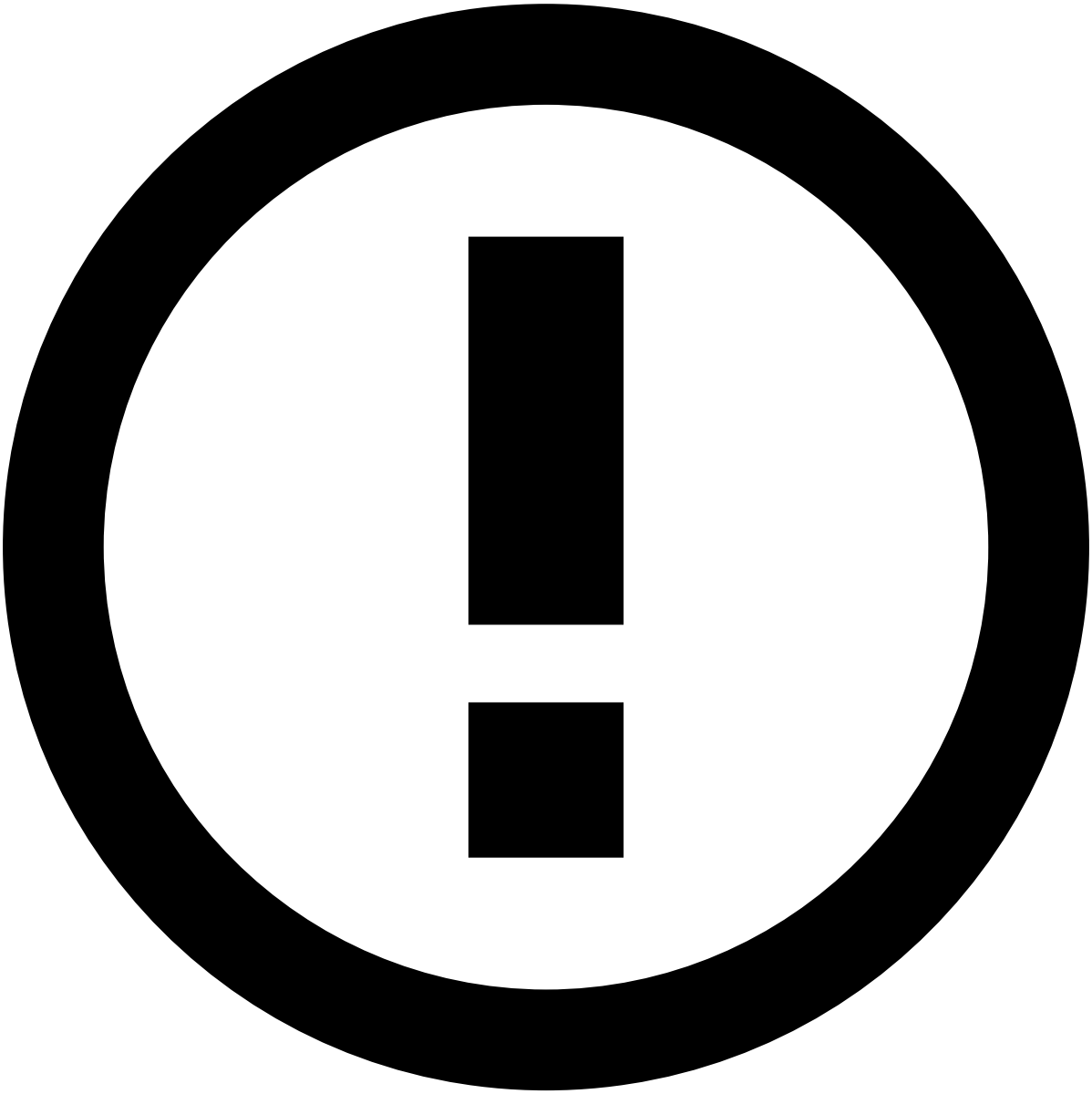
Configuring an SSL Certificate for Pulse



Must change the default certificate

Pulse bundles a self-signed certificate which allows users to connect to it through HTTPS. This can be found in the conf/engine/jetty/ directory, comprising a keystore and a jetty.xml file. This certificate **must not be used in production** environments, as it is insecure.

In this page you can learn how to set up a certificate for SSL in Pulse.



info

This section describes how to create a self-signed certificate and install it in Pulse for use in the Jetty server. See Component Encryption for instructions on securing all Pulse communications.

Creating your self-signed certificateâ□□

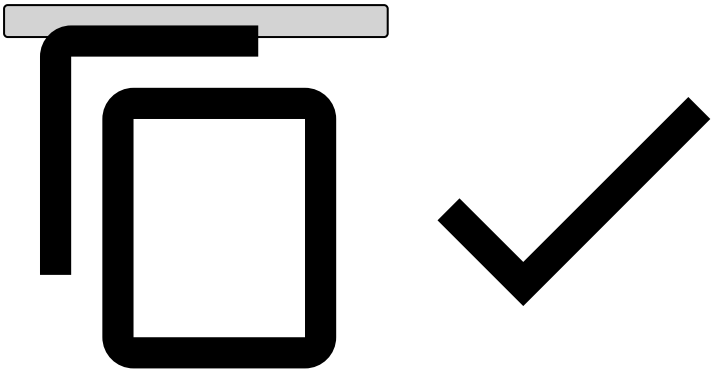
The **openssl** toolkit is used to generate an **RSA Private Key** and **CSR (Certificate Signing Request)**. It can also be used to generate self-signed certificates which can be used for testing purposes or internal usage.

The first step is to create your RSA Private Key. This key is a 1024 bit RSA key which is encrypted using Triple-DES and stored in a PEM format so that it is readable as ASCII text.

```
 $ openssl genrsa -des3 -out server.key 1024

Generating RSA private key, 1024 bit long modulus
.....................................................++++++
........++++++
```

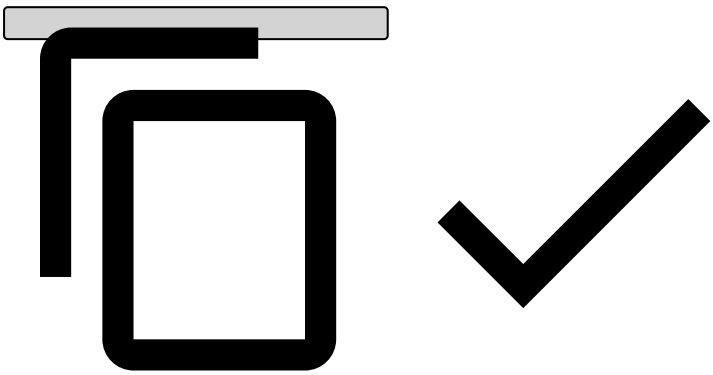
```
e is 65537 (0x10001)
Enter PEM pass phrase:
Verifying password - Enter PEM pass phrase:
```



Once the private key is generated a Certificate Signing Request can be generated. The CSR is then used in one of two ways. Ideally, the CSR will be sent to a Certificate Authority, such as Thawte or Verisign who will verify the identity of the requestor and issue a signed certificate. **The second option is to self-sign the CSR, which will be demonstrated in the next section**.

During the generation of the CSR, you will be prompted for several pieces of information. These are the X.509 attributes of the certificate. One of the prompts will be for "Common Name (e.g., YOUR name)". It is important that this field be filled in with the fully qualified domain name of the server to be protected by SSL. If the website to be protected will be https://acme.org, then enter acme.org at this prompt. The command to generate the CSR is as follows:

```
 $ openssl req -new -key server.key -out server.csr
Country Name (2 letter code) [XX]:US
State or Province Name (full name) []:California
Locality Name (eg, city) [Default City]:San Francisco
Organization Name (eg, company) [Default Company Ltd]:Acme
Organizational Unit Name (eg, section) []:IT
Common Name (eg, your name or your server's hostname) []:acme.org
Email Address []:[email protected]
Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:
An optional company name []:
```

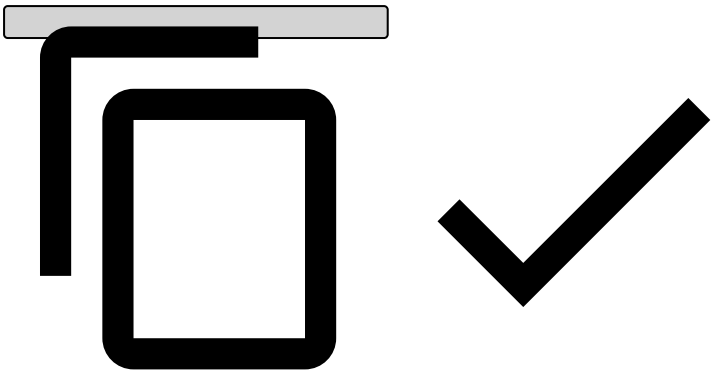


At this point you will need to generate a self-signed certificate because you either don't plan on having your certificate signed by a CA, or you wish to test your new SSL implementation while the CA is signing your certificate. This temporary certificate will generate an error in the client browser to the effect that the signing certificate authority is unknown and not trusted.

To generate a temporary certificate which is good for 365 days, issue the following command:

```
 $ openssl x509 -req -days 365 -in server.csr -signkey server.key -out server.crt
Signature ok
subject=/C=US/ST=California/L=San Francisco/O=Acme/OU=IT/CN=acme.org/[email protected]
```

```
Getting Private key
Enter pass phrase for server.key:
```

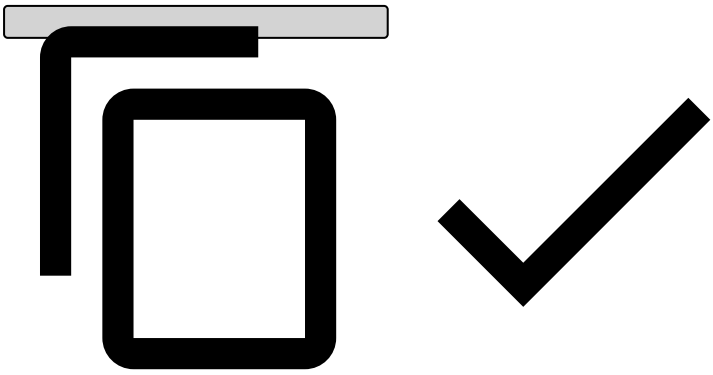


Add your certificate to Pulseâ□□

Once you have your certificate generated (self signed, or signed by an authorized CA), you can import your certificate into the embedded Jetty web server of Pulse.

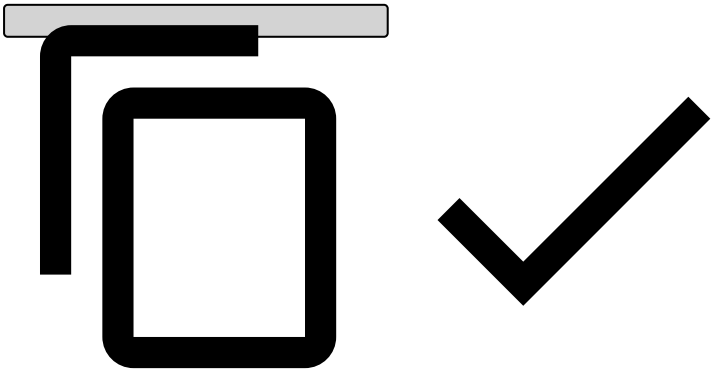
First you need to generate a password to encrypt your keystore:

```
 $ apg -n 1 -m 32
PhrirrAyfluwawkofNaljamBytbiaken
```



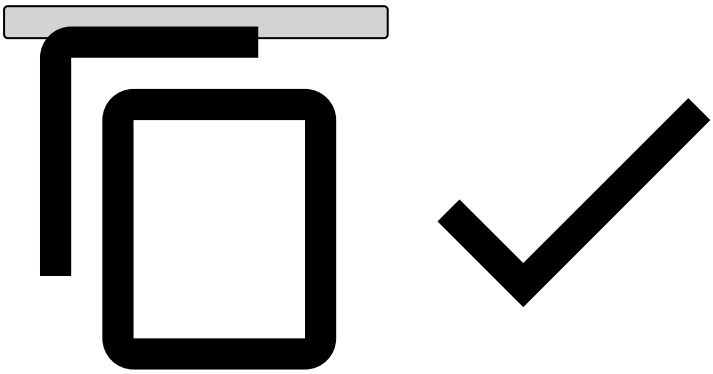
If you have a key and certificate in separate files, you need to combine them into a PKCS12 format file to load into a new keystore. The certificate can be one you generated yourself or one returned from a CA in response to your CSR.

```
 $ openssl pkcs12 -inkey server.key -in server.crt -export -out server.pkcs12
Enter pass phrase for server.key: (the password of your certificate)
Enter Export Password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
Verifying - Enter Export Password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
```



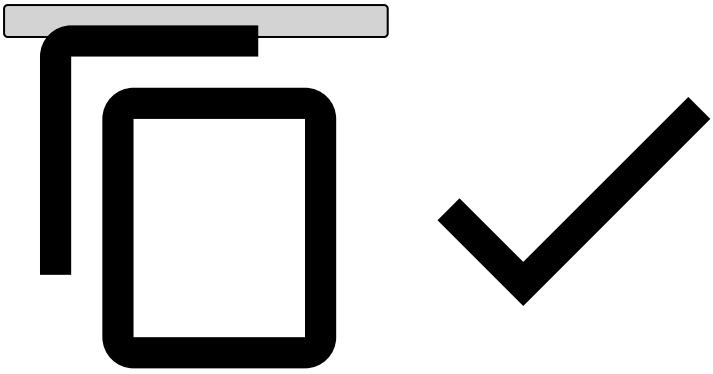
Now lets create the keystore where Pulse will read the certificate:

```
 $ $JAVA_HOME/bin/keytool -importkeystore -srckeystore server.pkcs12 -srcstoretype PKCS12 -
destkeystore keystore
Enter destination keystore password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
Re-enter new password: (the new password generated to protect the pkcs12 file, i.e.
PhrirrAyfluwawkofNaljamBytbiaken)
Enter source keystore password: (the password of your certificate)
Entry for alias 1 successfully imported.
Import command completed:  1 entries successfully imported, 0 entries failed or cancelled
```



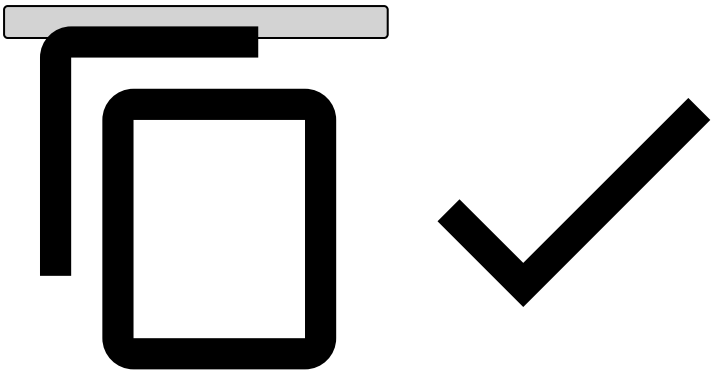
Now copy the resulting file **keystore** to `$PULSE_HOME/conf/engine/jetty/keystore`:

```
$ cp keystore $PULSE_HOME/conf/engine/jetty/keystore
```



Finally edit your `$PULSE_HOME/conf/engine/jetty/jetty.xml`:

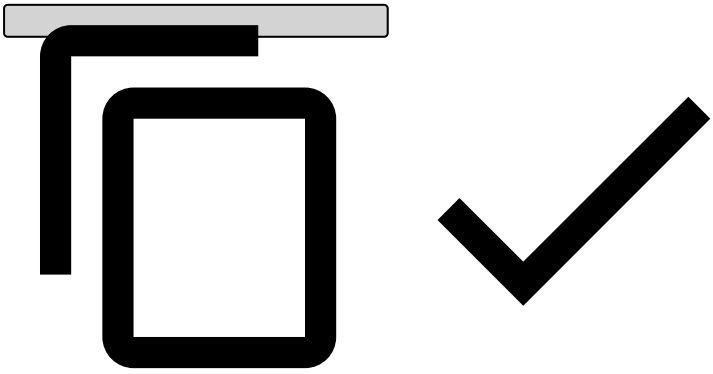
```
 <New id="sslContextFactory" class="org.eclipse.jetty.http.ssl.SslContextFactory">
  <Set name="KeyStore">conf/engine/jetty/keystore</Set>
  <Set name="KeyStorePassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
  <Set name="KeyManagerPassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
  <Set name="TrustStore">conf/engine/jetty/keystore</Set>
  <Set name="TrustStorePassword">PhrirrAyfluwawkofNaljamBytbiaken</Set>
</New>
```



Enforce SSL Configurationâ□□

Define the following line in the pulse.properties file:

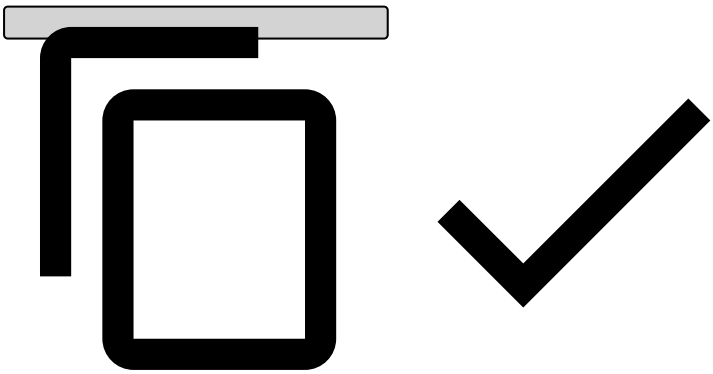
```
pulse.manager.enforcessl=true
```



Configure SSL in FIPS modeâ□□

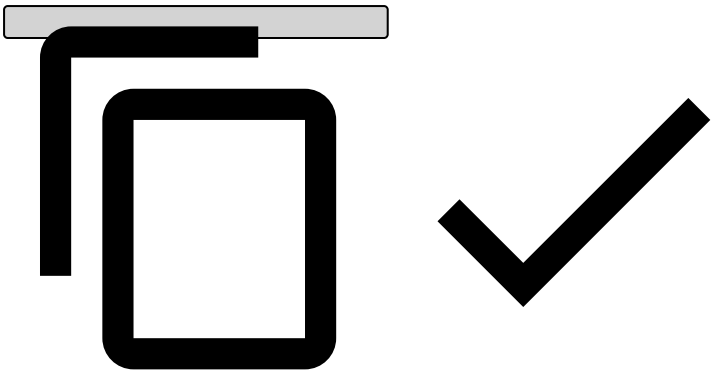
Running Pulse in a FIPS (Federal Information Processing Standards) compliant system, and with a keystore certificate that contains aliases, wildcards, or multiple hosts, may result in the following error when starting the server:

```
java.security.KeyManagementException: FIPS mode: only SunJSSE KeyManagers may be used
```



To configure Pulse to run in FIPS mode, edit the Jetty configuration at `$PULSE_HOME/conf/engine/jetty/jetty.xml`, and change the `sslContextFactory` definition as follows:

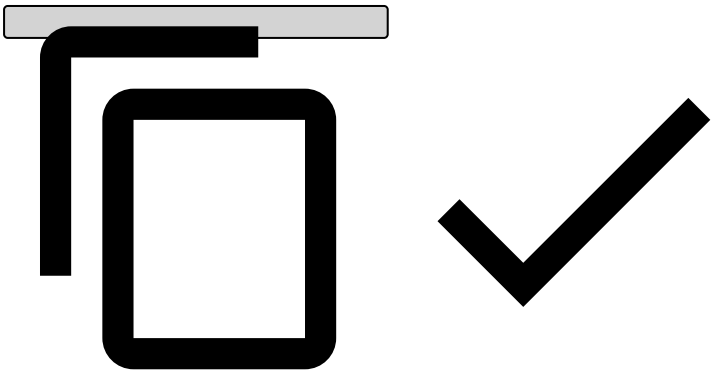
```
 - <New id="sslContextFactory" class="org.eclipse.jetty.http.ssl.SslContextFactory">
+ <New id="sslContextFactory"
class="com.feedzai.pulse.service.webapps.jetty.FIPSSslContextFactory">
```



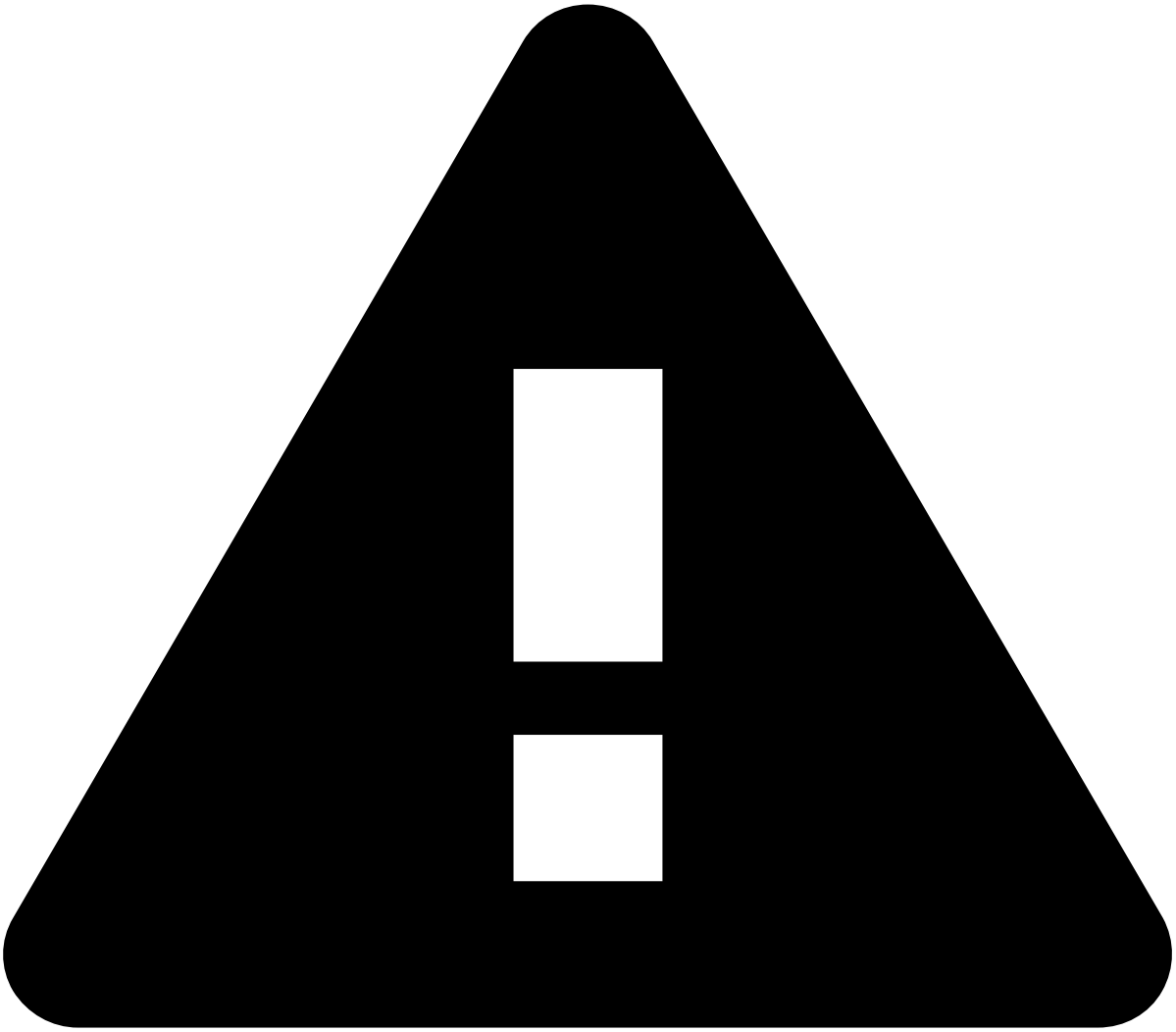
Depending on the used database, a similar error may prevent the connection from Pulse to the database. This as been observed when trying to connect to PostgreSQL using the PostgreSQL JDBC Driver. In such cases, it may be required to explicitly configure the driver to use the appropriate SSL factory.

For PostgreSQL, this can be achieved by configuring the JDBC connection URL to explicitly specify the SSL factory to be used:

```
 pulse.global.database.jdbc=jdbc:postgresql://<HOST:PORT>/<DATABASE>?
ssl=true&sslfactory=org.postgresql.ssl.DefaultJavaSSLFactory
```



The above configuration will use the default Java SSL factory provided by the JVM, rather than the PostgreSQL JDBC driver's custom SSL implementation. This ensures that the database connection uses the FIPS-compliant security providers from the Java runtime environment.



warning

This configuration disables the certificate aliases and SNI (Server Name Indication). As of now, there are no alternatives to run Pulse with FIPS and certificate aliases, or SNI.